



Bloc 4

Maintenir l'application logicielle en condition opérationnelle

Dossier écrit - Projet Spity

	Expert en développement logiciel
	Dorian Joly
	Ynov 2024 - C4.1.1 à C4.3.3
	1.0 - 13 août 2026
	7 compétences documentées et vérifiables

Mise en situation réelle et fictive déclarée - preuves anonymisées et reproductibles

Sommaire

Sommaire	2
Déclaration de portée	4
1. Résumé exécutif	5
2. Correspondance avec le référentiel	6
2.1 Revue transversale de clôture	6
2.2 Comment lire et évaluer chaque compétence	6
3. Contexte technique et responsabilités	8
3.1 Architecture maintenue	8
3.2 Rôles	8
4. C4.1.1 - Gérer les mises à jour des dépendances	9
4.0 Ce que la compétence demande et pourquoi elle est nécessaire	9
4.1 Processus précis	9
4.2 Mise à jour réalisée	9
4.3 Sécurisation	9
4.4 Déroulé opérationnel d'une mise à jour	9
4.5 Comment la compétence est démontrée	10
5. C4.1.2 - Concevoir la supervision et l'alerte	11
5.0 Ce que la compétence demande et la réponse retenue	11
5.1 Sondes et finalité	11
5.2 Disponibilité, couverture et signalement	11
5.3 Résultat observé et exercice	11
5.4 Du signal à la décision de maintenance	11
5.5 Ce que le jury peut contrôler	12
6. C4.2.1 - Consigner les anomalies	13
6.0 Ce que la compétence demande et le principe appliqué	13
6.1 Collecte et cycle de vie	13
6.2 Anomalies réelles et vérification	13
6.3 Démonstration détaillée du traitement d'une anomalie	13
6.4 Contrôles, preuves et critères de qualité	14
7. C4.2.2 - Créer et déployer un correctif via CI/CD	15
7.0 Ce que la compétence demande et la difficulté traitée	15
7.1 Anomalie traitée et décision	15

7.2 Correctif mis en place	15
7.3 Intégration et déploiement continu	15
7.4 Vérification reproductible	15
7.5 Retour arrière	16
7.6 Chaîne de correction expliquée pas à pas	16
7.7 Lecture des preuves et limite de portée	16
8. C4.3.1 - Proposer des améliorations	17
8.0 Ce que la compétence demande et la méthode d'arbitrage	17
8.1 Backlog mesurable et arbitrage	17
8.2 Indicateurs et retours utilisateurs	17
8.3 Cadence, responsabilité et décision	17
8.4 Comment une recommandation devient une décision exploitable	17
8.5 Preuves à relier à l'attendu	18
9. C4.3.2 - Établir le journal des versions	19
9.0 Ce que la compétence demande et le problème de traçabilité	19
9.1 Source de vérité et statuts	19
9.2 Correctifs et évolutions documentés	19
9.3 Cadence, responsabilité et contrôle	19
9.4 Cycle de vie d'une version et lecture devant le jury	19
9.5 Preuves et critères de qualité	20
10. C4.3.3 - Collaborer avec le support	21
10.0 Ce que la compétence demande et l'organisation retenue	21
10.1 Registre et cycle de collaboration	21
10.2 Cas de collaboration présenté	21
10.3 Vérification du protocole et confidentialité	21
10.4 Déroulé de résolution et éléments à présenter	21
11. Protection des données et sécurité des preuves	22
12. Conclusion	23
13. Index des annexes	24

Certification : Expert en développement logiciel

Projet : Spity, réseau social pour la communauté de l'escalade

Candidat : Dorian Joly

Version du dossier : 1.1

Date : 13 août 2026

Référentiel : Ynov 2024, pages 15 à 17

Déclaration de portée

Ce dossier présente la maintenance de Spity, application Next.js 16 avec MariaDB, conteneurs Docker et GitHub Actions. Il couvre la gestion des dépendances, la supervision, la consignation et la correction des anomalies, les recommandations, le journal des versions et la collaboration support.

Les preuves sont de trois natures : sources versionnées, états externes publics figés en JSON et mise en situation fictive autorisée par le référentiel. Les simulations sont toujours annoncées comme telles. Aucun incident réel, échange humain ou déploiement n'est inventé. Aucun LXC n'a été utilisé pour constituer le dossier.

Lecture de remise : le répertoire `dossier-jury/` organise ce dossier, la revue finale, les commandes et les preuves pour une présentation progressive au jury.

1. Résumé exécutif

Spity dispose d'une chaîne de maintenance reproductible : Dependabot surveille chaque semaine npm et GitHub Actions, la CI bloque lint, types, tests, audit, build, MariaDB, accessibilité, Lighthouse et recettes Playwright, tandis qu'une sonde externe vérifie la production. Les images de release sont immuables et associées à une version, une révision et un manifeste.

Le 13 août 2026, l'état vérifié est le suivant :

- 0 vulnérabilité de production et 0 alerte haute/critique dans l'audit complet après mise à jour des outils ;
- 152 tests Jest, 43 tests de maintenance, 11 scénarios MariaDB et 6 recettes Playwright ;
- 10 pages authentifiées sur 10 à 100 % Lighthouse accessibilité ;
- CI complète verte sur e3784b7 ;
- 30 exécutions de supervision terminées dans l'échantillon public de 39 runs, toutes réussies, avec production ok ;
- version observée en production : 0.1.0-jury, révision 49c4ea0 ;
- dérive de déploiement réelle consignée : la production reste 19 commits derrière la référence auditée.

La dérive n'a pas été corrigée par un déploiement non autorisé. Elle est transformée en action de release contrôlée, avec sauvegarde et retour arrière.

2. Correspondance avec le référentiel

Code compétence	Attendu principal	Réponse Spity	Preuves majeures	Statut
C4.1.1	Processus précis : fréquence, périmètre, type	Cadence hebdomadaire et mensuelle, politique exécutable, audit planifié, SBOM, revue PR et lot réel qualifié	spity/MAINTENANCE.md, C411-01 à C411-03	Industrialisé et vérifié
C4.1.2	Supervision adaptée, sondes, critères qualité/performance, disponibilité	Politique versionnée, contrôle 15 min, qualification S1/S2/S3, artefacts 90 jours, incident/rétablissement unique et SLO 30 jours avec garde de couverture	spity/OBSERVABILITY.md, C412-01 à C412-04	Industrialisé et vérifié
C4.2.1	Collecte structurée, fiche reproductible, analyse et préconisations	Registre versionné, machine à états, confidentialité contrôlée, formulaires, CI dédiée et deux anomalies réelles	spity/INCIDENT_MANAGEMENT.md, C421-01 à C421-04	Industrialisé et vérifié
C4.2.2	Correctif décrit utilisant intégration/déploiement continu	Contrôle de promotion version/révision, staging CI, candidate de release, rapport conservé, exercice reproductible et staging vérifié	spity/RELEASE_VERIFICATION.md, C422-01 à C422-04	Industrialisé et vérifié
C4.3.1	Recommandations réalistes, argumentées, coûts/délais/gains	Registre mesurable, indicateurs, coûts/délais, retours qualifiés, revue mensuelle et CI dédiée	spity/IMPROVEMENT_MANAGEMENT.md, C431-01 à C431-03	Industrialisé et vérifié
C4.3.2	Journal des versions et correctifs déployés	Registre versionné, identité SemVer/SHA, correctifs documentés, preuve de santé et revue mensuelle	spity/RELEASE_JOURNAL.md, C432-01 à C432-03	Industrialisé et vérifié
C4.3.3	Problème résolu avec contexte, résolution et contributions	Registre contrôlé de transmissions support/mainteneur, critères fonctionnels, expertise technique, confidentialité et validation simulée déclarée	spity/SUPPORT.md, C433-01 à C433-03	Industrialisé et vérifié

2.1 Revue transversale de clôture

La revue REVUE_FINALE_BLOC_04.md aligne, pour chaque compétence, l'attendu du référentiel, le mécanisme Spity, la commande de contrôle et la preuve à présenter. `npm run bloc4:check` vérifie ces sept lignes ensemble : statut dans le dossier, résultat dans le plan, sources opérationnelles, assertions sur les preuves, registres vivants et SHA-256 du manifeste. Cette porte est incluse dans la qualité CI ; elle ne contacte aucun environnement externe.

2.2 Comment lire et évaluer chaque compétence

Les sections 4 à 10 suivent la même grille afin que le lecteur puisse vérifier le raisonnement sans devoir deviner le rôle d'un fichier. Chaque compétence présente :

- **l'attendu** : reformulation du critère évalué et du risque métier ou technique associé ;
- **la mise en œuvre** : politique, registre, script, workflow ou procédure réellement présents dans le dépôt ;
- **un scénario** : situation concrète, décision prise et résultat obtenu ou statut réellement ouvert ;
- **la vérification** : commande qui relit les données ou exercice qui valide aussi des cas d'échec ;

- **les preuves** : fichiers datés, sources de vérité et liens de traçabilité à présenter ;
- **la limite** : ce que la preuve ne permet pas d'affirmer, notamment lorsqu'il s'agit d'une simulation ou d'un staging.

Cette organisation rend le dossier utilisable à trois niveaux : lecture synthétique par la revue finale, lecture explicative par ce document, puis audit approfondi par les sources et preuves versionnées.

3. Contexte technique et responsabilités

3.1 Architecture maintenue

Le dépôt sépare l'application dans `spity/`, la CI/CD dans `.github/workflows/` et les documents dans `docs/`. L'application utilise Next.js 16.3, React 19, TypeScript, Drizzle ORM et MariaDB 11.4. Docker Compose isole l'application, la migration et la base. La production publique expose une route de santé sans secret.

La chaîne de livraison est structurée ainsi :

- modification du code ou du lockfile ;
- commit Git et push SSH ;
- qualité et recettes parallèles dans la CI ;
- construction d'images par SHA sur la branche d'intégration ;
- tag SemVer pour déclencher la release ;
- validation, smoke test, images candidates, manifeste et bundle ;
- promotion de production avec contrôle de version/révision.

3.2 Rôles

Le mainteneur est responsable de la qualification technique, des tests, des correctifs et du déploiement. GitHub Actions exécute les contrôles automatiques. Dependabot propose les mises à jour. Le product owner décide de la priorité métier et peut tenir le rôle support niveau 1 dans la configuration actuelle. Un utilisateur pilote valide les parcours lorsque ce rôle existe.

4. C4.1.1 - Gérer les mises à jour des dépendances

4.0 Ce que la compétence demande et pourquoi elle est nécessaire

Gérer les dépendances ne consiste pas à lancer une mise à jour automatique. Le référentiel attend un processus explicite qui indique **quand** les composants sont revus, **ce qui** est inclus dans le périmètre et **comment** chaque type de changement est qualifié. Pour Spity, le risque principal est double : conserver un composant vulnérable trop longtemps, ou introduire une régression en appliquant une mise à jour incompatible sans analyse.

La réponse retenue sépare donc la détection, la décision et l'intégration. Dependabot signale les évolutions ; la politique de dépendances définit les règles acceptables ; la CI vérifie le lot choisi ; le lockfile et le SBOM permettent ensuite de retrouver exactement ce qui a été installé. Une alerte ne devient jamais une modification de production sans revue et sans tests.

4.1 Processus précis

Dependabot analyse les dépendances npm chaque lundi à 06:00 et les actions GitHub à 06:30, fuseau Europe/Paris. Les mises à jour de production et de développement sont regroupées séparément et ciblent develop. Une revue manuelle mensuelle complète cette automatisation avec npm outdated, npm audit, les images Docker et les notes de version.

Les mises à jour de sécurité de production sont prioritaires. Les mineures compatibles sont groupées. Les majeures sont isolées avec analyse de rupture, recette ciblée et retour arrière. Le périmètre couvre npm, actions GitHub, Node.js, navigateurs CI et images ; il exclut toute modification automatique de secrets ou de données.

4.2 Mise à jour réalisée

L'audit du 13 août a trouvé des alertes hautes dans l'outillage Lighthouse/Puppeteer. Le lot a mis à niveau Lighthouse 12.6.1 vers 13.4.1, Puppeteer Core 24.43.1 vers 25.6.0, Playwright 1.61.1 vers 1.62.1, ESLint Next vers 16.3 et les types Node vers la branche 22.

Le résultat attendu est atteint : aucune alerte haute/critique dans l'audit complet et aucune vulnérabilité de production. Quatre alertes modérées restent liées à Drizzle/esbuild. La correction npm audit fix --force est rejetée car elle propose une rétrogradation cassante. La dérogation possède maintenant un propriétaire et une expiration, vérifiés automatiquement. La tentative de mise à jour de @hookform/resolvers a aussi révélé un conflit Ajv rendant le SBOM invalide ; la version 5.2.2 est verrouillée jusqu'au 13 octobre 2026 et le contrôle échoue si ce verrou ou son échéance dérive.

La commande npm run dependencies:check exécute les deux audits, applique dependency-policy.json, inventorie les versions en retard et produit un SBOM CycloneDX. Elle tourne chaque semaine dans dependency-maintenance.yml, conserve ses artefacts 90 jours et ouvre un ticket unique en cas d'échec. dependency-review.yml bloque en pull request toute nouvelle dépendance vulnérable de sévérité modérée ou supérieure. La preuve C411-03 conserve l'évaluation conforme sous Node 22, le SHA-256 du lockfile et les métadonnées du SBOM.

4.3 Sécurisation

Le lockfile garantit les versions transitives et son SHA-256 est figé dans la preuve. La CI rejoue l'ensemble des contrôles et bloque la promotion en cas d'échec. La procédure de retour arrière utilise les images immuables et la sauvegarde créée avant migration.

4.4 Déroulé opérationnel d'une mise à jour

Le mainteneur suit le déroulé suivant, qui peut être expliqué et rejoué devant le jury :

- **Détecter** : Dependabot ou la revue mensuelle remonte une version, une alerte ou une information de rupture.

- **Qualifier** : la dépendance est classée selon son périmètre (production, développement, CI, image), son type de changement (correctif, mineure, majeure) et son niveau de risque.
- **Décider** : une mise à jour compatible est regroupée ; une majeure ou une correction avec rupture reste isolée ; une impossibilité technique devient une exception documentée avec échéance.
- **Vérifier** : les audits, le contrôle de politique, le SBOM, le lint, le typage, les tests et le build sont rejoués.
- **Tracer** : la décision, le SHA du lockfile, les résultats et les éventuelles exceptions sont conservés dans les preuves et le changelog.
- **Promouvoir ou revenir** : le lot ne progresse que si les portes CI sont vertes ; sinon il est corrigé ou annulé sans forcer le gestionnaire de paquets.

Cette séquence protège aussi contre une solution trompeuse : `npm audit fix --force` peut réduire un nombre d'alertes tout en introduisant une rétrogradation ou une rupture. Spity refuse ce raccourci et documente la décision lorsqu'une correction automatique est inadaptée.

4.5 Comment la compétence est démontrée

Élément attendu	Élément présenté au jury	Ce que cela permet de vérifier
Fréquence et périmètre	MAINTENANCE.md, configuration Dependabot et workflows dédiés	Les revues sont planifiées et couvrent npm, Actions, Node, navigateurs CI et images.
Typologie des mises à jour	Politique et décision C411-02	Les correctifs, mineures, majeures et exceptions ne suivent pas la même règle.
Qualification technique	<code>npm run dependencies:check</code> et C411-03	Les audits, versions, verrou, SBOM et exceptions sont cohérents.
Traçabilité	Lockfile, SHA-256, SBOM et manifeste	L'état analysé peut être relié à une révision et contrôlé à nouveau.

La commande à exécuter est `npm run dependencies:check`. La preuve B4-C411-03-contrôle-dependances-2026-08-13.json est la sortie de référence ; elle ne masque pas les alertes modérées restantes et explique leur traitement.

5. C4.1.2 - Concevoir la supervision et l'alerte

5.0 Ce que la compétence demande et la réponse retenue

La supervision attendue ne se résume pas à vérifier qu'une URL répond. Elle doit observer un service avec des critères adaptés, signaler les situations réellement actionnables et mesurer la disponibilité sans confondre une mesure incomplète avec une panne. Spity distingue donc la santé applicative, l'identité de la version exécutée, la latence et la couverture des observations.

La stratégie choisie combine une route de santé publique sans donnée sensible, une politique de supervision versionnée, des workflows planifiés, un calcul de SLO et un exercice local qui rejoue les erreurs. Elle transforme un signal brut en décision : incident de disponibilité, anomalie de contrat ou investigation de performance.

5.1 Sondes et finalité

La route `/api/health` confirme l'état de l'application, la connexion MariaDB, la version et la révision. `monitoring-policy.json` versionne la cadence de 15 minutes, le timeout de 15 secondes, les deux reprises, le seuil de 3 000 ms et l'objectif de disponibilité de 99,5 % sur 30 jours.

Contrôle	Qualification	Décision
HTTP, réseau, timeout, JSON ou statut applicatif	S1, impact disponibilité	Issue d'incident unique, investigation et rétablissement vérifié.
Métadonnées absentes	S2, contrat de supervision invalide	Vérification de la version/révision et correction prioritaire.
Latence ou révision inattendue	S3, disponible mais dégradé	Action de performance ou de contrôle de déploiement, sans faux calcul d'indisponibilité.

5.2 Disponibilité, couverture et signalement

Le workflow de sonde conserve chaque rapport JSON 90 jours. Un deuxième workflow calcule chaque jour le SLO à partir des seuls runs **planifiés**, exclut les exercices manuels et mesure disponibilité, échantillons attendus, couverture et données exclues. Une fenêtre incomplète (< 96 observations ou < 95 % de couverture) est insuffisant-data et n'ouvre pas de faux incident. Une vraie brèche ouvre ou actualise une issue SLO unique ; une mesure compliant la ferme. Les rapports et issues ne contiennent ni secret, ni donnée personnelle, ni réponse brute.

5.3 Résultat observé et exercice

La collecte publique a trouvé 30 runs de supervision terminés dans les 39 plus récents, tous réussis, et la production répond ok. Le calcul SLO actuel mesure 18 observations planifiées sur les 96 minimales : il reste donc insuffisant-data sans ouvrir de fausse alerte. Il est testé sur une fenêtre couverte, une brèche réellement alertable, une couverture insuffisante et l'exclusion d'un déclenchement manuel. L'exercice local contrôlé couvre désormais un cas sain, un HTTP 503/applicatif avec deux tentatives et une latence S3 encore disponible, sans toucher à la production.

5.4 Du signal à la décision de maintenance

Le cycle de supervision est volontairement explicite :

- le workflow planifié appelle `/api/health` avec le timeout et les reprises prévus ;
- le script normalise la réponse en rapport JSON sans conserver le corps brut ni des informations sensibles ;
- la politique qualifie le résultat en S1, S2 ou S3 ;

- une erreur S1 crée ou met à jour un incident unique afin d'éviter les doublons ;
- le calcul SLO quotidien utilise seulement les observations planifiées et mesure la couverture de la fenêtre ;
- une mesure redevenue conforme permet de clôturer l'alerte, tandis qu'une couverture insuffisante reste explicitement non concluante.

Ce fonctionnement répond à deux risques fréquents. Le premier est le faux positif : un déclenchement manuel ou un échantillon trop court ne doit pas être traité comme une indisponibilité réelle. Le second est le faux négatif : une application qui répond mais ne fournit plus sa version ou sa révision ne respecte plus le contrat de supervision et doit être investiguée.

5.5 Ce que le jury peut contrôler

Élément attendu	Mécanisme concret	Preuve ou commande
Sonde adaptée au service	Route santé, timeout, reprises, seuil de latence et classification S1/S2/S3	OBSERVABILITY.md, monitoring-policy.json, <code>npm run monitoring:probe</code>
Disponibilité mesurée	SLO 30 jours à 99,5 %, couverture minimale et exclusion des runs manuels	<code>npm run monitoring:slo</code> , preuve C412-04
Alerte exploitable	Incident unique, historique et règle de rétablissement	Workflows de supervision et preuve C412-03
Cas d'échec testé	503, erreur applicative, reprises et latence dégradée	<code>npm run bloc4:exercise</code>

Les preuves C412-01 à C412-04 permettent de séparer l'historique de supervision, la santé observée, l'exercice d'alerte et le calcul SLO. Ainsi, un résultat sain, une simulation et une mesure de couverture ne sont jamais mélangés dans une même affirmation.

6. C4.2.1 - Consigner les anomalies

6.0 Ce que la compétence demande et le principe appliqué

Consigner une anomalie signifie rendre le problème compréhensible et rejouable par une autre personne. Une simple issue ou une phrase du type « cela ne fonctionne pas » ne permet ni d'établir l'impact, ni de choisir une correction, ni de vérifier la résolution. Spity utilise donc un registre structuré qui sépare le signal initial, l'analyse, la décision, l'action et la vérification.

La fiche d'incident n'est pas une archive figée : elle porte un cycle de vie contrôlé. Chaque changement d'état correspond à une action réellement effectuée. Cela évite de clôturer un défaut avant investigation ou de faire disparaître une anomalie parce qu'un correctif paraît probable.

6.1 Collecte et cycle de vie

Une issue GitHub recueille le signal initial ; elle impose date ISO, impact, version/révision, reproduction, preuves anonymisées et confirmation de confidentialité. Après triage, le mainteneur crée une fiche SPITY-INC-YYYY-NNNN versionnée dans `spity/incidents/`. Cette fiche est la source de vérité : sévérité, priorité, propriétaire, environnement, impact, préconditions, observé, attendu, cause, décision, actions, vérification et preuves y sont séparés.

Le cycle ordonné passe par `reported`, `triaged`, `investigating`, `planned`, `resolving`, `validating`, `resolved` puis `closed`. Il est contrôlé par `incident-policy.json`. Les chemins de rejet et doublon restent possibles ; une fiche planifiée reste explicitement ouverte. Le script `check-incident-registry.mjs` refuse une transition interdite, un historique non chronologique, une clôture sans vérification, un identifiant dupliqué, une preuve absente ou un motif de secret, jeton, adresse privée, e-mail ou clé privée.

6.2 Anomalies réelles et vérification

SPITY-INC-2026-0001 reprend l'échec d'accessibilité authentifiée : l'état vide était reproductible sur une base vierge et sur des surfaces claire/sombre. La cause racine est l'héritage de couleurs par un composant partagé ; la décision a consisté à le rendre autonome, avec validation Lighthouse, Playwright et CI. Elle est clôturée sans prétendre que sa promotion production a eu lieu.

SPITY-INC-2026-0002 consigne la dérive observée entre production et référence audité. Elle reste `planned` : la décision est de préparer une release contrôlée, jamais de déployer uniquement pour fermer un écart documentaire. L'audit courant accepte les deux fiches ; l'exercice contrôlé refuse une clôture directe de `reported` vers `closed` et un jeton Bearer simulé, uniquement en mémoire.

6.3 Démonstration détaillée du traitement d'une anomalie

Le cas SPITY-INC-2026-0001 montre le cycle complet sur un problème d'accessibilité. Le signal fonctionnel identifie des états vides difficilement lisibles après authentification. La reproduction précise le contexte de base vierge, les surfaces concernées et le résultat attendu. L'investigation technique relie le défaut à l'héritage de couleur du composant partagé `EmptyState` ; ce diagnostic est différent du constat initial afin que la partie fonctionnelle et la cause technique restent toutes deux lisibles.

Le correctif rend le composant autonome. La validation ne repose pas sur une seule capture : Lighthouse contrôle l'accessibilité des pages authentifiées, Playwright rejoue le parcours et la CI vérifie l'ensemble. La clôture de la fiche porte donc une décision, des actions, des preuves et un résultat. Elle ne revendique pas de déploiement production, puisque cette information ne fait pas partie de la preuve disponible.

Le cas SPITY-INC-2026-0002 apporte la situation inverse : le problème est réel et documenté, mais la résolution n'est pas encore autorisée. Son statut `planned` est conservé. Cette distinction prouve que le registre représente l'état réel, y compris lorsqu'une action reste à réaliser.

6.4 Contrôles, preuves et critères de qualité

Point contrôlé	Règle appliquée	Intérêt pour la maintenance
Identification	Identifiant stable SPITY-INC-YYYY-NNNN et absence de doublon	Retrouver une anomalie dans les preuves, le support et le journal de release.
Reproductibilité	Préconditions, observé, attendu, environnement et impact séparés	Permettre à un autre mainteneur de comprendre et rejouer le défaut.
Cycle de vie	Transitions ordonnées contrôlées par politique	Empêcher une clôture arbitraire ou une chronologie incohérente.
Vérification	Preuve obligatoire avant c losed	Distinguer un correctif supposé d'une résolution vérifiée.
Confidentialité	Rejet des secrets, jetons, e-mails et adresses privées	Conserver un dossier consultable sans exposer de donnée sensible.

Les commandes `npm run incidents:check` et `npm run incidents:exercise` donnent respectivement l'audit du registre réel et la démonstration de ses rejets. Les preuves C421-01 à C421-04 présentent les deux fiches, la sortie de registre et l'exercice contrôlé.

7. C4.2.2 - Créer et déployer un correctif via CI/CD

7.0 Ce que la compétence demande et la difficulté traitée

Cette compétence demande de montrer qu'un correctif est décrit, intégré et déployé par une chaîne continue. Le risque n'est pas seulement qu'un test échoue : une application peut être disponible tout en exécutant un binaire qui ne correspond pas à celui qui a été validé. Spity traite ce risque avec une porte de promotion qui compare l'identité attendue du candidat avec l'identité réellement exposée par sa route de santé.

Le dossier ne prétend pas qu'une réussite CI est une production mise à jour. Il démontre un correctif de chaîne CI/CD, un staging réellement vérifié et une procédure de promotion/retour arrière qui reste soumise à l'autorisation de release.

7.1 Anomalie traitée et décision

SPITY-INC-2026-0002 a révélé une dérive réelle : la santé publique exposait une révision de production différente de la référence audité. La disponibilité restait correcte, mais la chaîne ne transformait pas l'identité attendue de l'artefact en porte de promotion exécutable. La décision retenue est de corriger ce risque sans déployer la production pour les besoins du dossier.

7.2 Correctif mis en place

Le correctif `spity/scripts/verify-deployment.mjs` réutilise le contrat de santé et exige désormais trois informations avant qu'un candidat soit accepté : l'URL de santé, la version attendue et la révision Git complète attendue. Une différence de version ou de SHA produit un échec `S3/deployment-verification`, un rapport JSON et bloque la promotion. Un candidat cohérent doit renvoyer `status: ok` ainsi que les deux valeurs exactes.

La règle est volontairement stricte : une application disponible avec un mauvais binaire n'est pas assimilée à une release réussie. Le contrôle ne contacte aucun environnement de production par défaut et ne contient aucun secret.

7.3 Intégration et déploiement continu

Le workflow `Continuous integration` exécute ce correctif après le smoke test de staging sur l'image immuable `sha-$GITHUB_SHA`, avant l'arrêt de l'environnement et la publication des images. Le workflow `Release` applique la même vérification à l'image candidate avant toute promotion stable. Les rapports `deployment-verification.json` rejoignent les artefacts de staging conservés par GitHub.

Le bundle de release contient les scripts de contrôle et la procédure `DEPLOYMENT.md` les impose après le démarrage local du candidat. La production n'est donc pas déclarée mise à jour : elle reste soumise à l'autorisation de promotion, à la sauvegarde et aux vérifications post-déploiement prévues.

7.4 Vérification reproductible

L'exercice `npm run bloc4:deployment-exercise` démarre uniquement un serveur HTTP local en mémoire. Il accepte un candidat conforme et refuse séparément une version `0.1.0` inattendue puis une révision Git différente. La preuve C422-03 conserve les valeurs attendues, observées, la classification et le résultat, sans Docker, LXC, base de données ou appel de production. La suite de maintenance actuelle couvre ce contrat en plus des autres portes Bloc 4.

La preuve C422-04 complète l'exercice par une exécution GitHub Actions réellement terminée avec succès sur `develop` : les portes qualité, MariaDB, Lighthouse et recette BC02 précèdent la construction d'images immuables, le smoke test de staging, le contrôle de version/révision, la publication d'images et l'artefact de déploiement. Elle documente le staging éphémère, jamais une promotion de production non observée.

7.5 Retour arrière

Le retour arrière remet le tag d'image immuable précédent, relance le même contrôle de version/révision et conserve les journaux. Une restauration MariaDB n'est réalisée que si une migration empêche l'ancienne application de fonctionner, avec validation explicite et sauvegarde contrôlée. Un rollback ne réécrit jamais l'historique Git : il est journalisé comme une nouvelle décision traçable.

7.6 Chaîne de correction expliquée pas à pas

La chaîne CI/CD est lue dans le sens suivant :

- une anomalie ouvre une décision de correction, ici la dérive entre identité attendue et identité observée ;
- le script `verify-deployment.mjs` formalise le contrat : URL de santé, version attendue et SHA complet attendu ;
- la CI construit une image immuable liée au commit, déploie un staging temporaire et exécute le smoke test ;
- le script compare les valeurs de santé du staging aux valeurs du candidat ;
- une différence produit un rapport d'échec et interdit la poursuite de la promotion ;
- un candidat cohérent produit un rapport conservé dans les artefacts et peut devenir une candidate de release ;
- une promotion production reste une décision distincte, accompagnée de sauvegarde, contrôle post-déploiement et journalisation.

Le scénario est testé localement sans environnement externe : le serveur HTTP en mémoire répond successivement avec les valeurs correctes, une version incorrecte puis une révision incorrecte. Les deux erreurs doivent échouer individuellement ; cela prouve que le contrôle ne se contente pas de vérifier un HTTP 200.

7.7 Lecture des preuves et limite de portée

Élément	Source présentée	Ce qui est effectivement démontré
Besoin de correction	SPITY-INC-2026-0002 et C422-01	La dérive d'identité est analysée et une décision est prise.
Contrat exécutable	<code>verify-deployment.mjs</code> et C422-02	Version et révision exactes sont obligatoires avant acceptation.
Cas conforme et erreurs	C422-03, <code>npm run bloc4:deployment-exercise</code>	Un faux candidat est refusé sans Docker, base ou production.
CI réelle	C422-04 et <code>workflow ci.yml</code>	Le staging sur <code>develop</code> a franchi les portes qualité, recette, MariaDB, Lighthouse et vérification de déploiement.
Continuité d'exploitation	<code>DEPLOYMENT.md</code> et procédure de rollback	Le retour arrière est décrit, contrôlé et tracé.

La limite est importante : C422-04 prouve un **staging éphémère validé**, jamais une promotion de production non observée. Cette précision protège la valeur du dossier : chaque preuve est utilisée pour l'affirmation exacte qu'elle permet de soutenir.

8. C4.3.1 - Proposer des améliorations

8.0 Ce que la compétence demande et la méthode d'arbitrage

Proposer une amélioration n'est pas produire une liste d'idées. Le référentiel attend des recommandations réalistes, expliquées et comparables : il doit être possible de comprendre le problème traité, le bénéfice attendu, l'effort, le délai, le risque et la manière de mesurer le résultat. Spity transforme ces éléments en fiches versionnées plutôt qu'en notes dispersées.

La priorisation repose sur des critères visibles. Une recommandation ne peut pas être déclarée terminée parce qu'elle est séduisante ou urgente : elle doit posséder un indicateur de départ, une cible, un responsable, un plan de retour arrière et une mesure de résultat. Cela permet au product owner et au mainteneur de partager une décision intelligible.

8.1 Backlog mesurable et arbitrage

Le backlog spity/improvements/ remplace la simple liste de recommandations par quatre fiches versionnées. Chaque fiche lie une source, un impact, une réduction de risque, une confiance, un effort, un coût en jours.homme, un délai, un rollback et des indicateurs de référence/cible. La formule $\text{impact} * 3 + \text{risque réduit} * 2 + \text{confiance} - \text{effort}$ classe les priorités actives et le contrôleur refuse un ordre contradictoire.

La priorité 1 livre des métriques persistantes avec une couverture p95 et disponibilité de 95 % ; la priorité 2 vise 70 % des contrats API critiques testés ; la priorité 3 permet de qualifier 100 % des futurs retours par zone, type de signal et bénéfice attendu ; la priorité 4 reste une étude Drizzle séparée, sans modification du lockfile ni rétrogradation automatique.

8.2 Indicateurs et retours utilisateurs

Les signaux opérationnels proviennent de la politique de supervision, des preuves CI et de la politique de dépendances. La seule source de retour non opérationnelle disponible est une simulation support explicitement déclarée : elle est utilisée pour concevoir la collecte, jamais comme un avis utilisateur réel. Le formulaire support collecte désormais la zone fonctionnelle, le type de signal et le résultat attendu, en plus du contexte déjà anonymisé.

Une source de retour contenant un e-mail, une adresse privée, un secret ou un jeton est refusée. L'exercice C431-03 vérifie le backlog sain, le refus d'un score volontairement erroné et le refus d'une adresse e-mail simulée. Il s'exécute uniquement en mémoire.

8.3 Cadence, responsabilité et décision

Chaque premier jour du mois, le workflow Improvement review valide le backlog, exécute les tests dédiés et conserve un rapport 90 jours. Le product owner arbitre priorité, coût et délai ; le mainteneur valide indicateurs, faisabilité et retour arrière ; le support ou pilote qualifie le bénéfice attendu. Une fiche ne peut être clôturée qu'avec un résultat mesuré : aucune recommandation approuvée n'est présentée comme déjà réalisée.

Cette boucle reste réaliste pour Spity : elle ne déclenche ni déploiement ni refonte, et elle laisse la promotion de production soumise à l'autorisation déjà définie par C4.2.2.

8.4 Comment une recommandation devient une décision exploitable

Chaque fiche suit un chemin reproductible :

- **Source du besoin** : signal de supervision, dette de dépendances, incident, résultat de CI ou retour qualifié.
- **Formulation** : problème, objectif, gain attendu et population ou zone concernée.
- **Évaluation** : impact, risque réduit, confiance, effort, coût, délai et dépendances éventuelles.

- **Priorisation** : application de la formule de score puis contrôle de l'ordre des priorités.
- **Décision** : arbitrage par le product owner avec validation de faisabilité et de rollback par le mainteneur.
- **Mesure** : comparaison d'un indicateur de référence avec une cible avant toute clôture.

Les quatre fiches présentes illustrent des catégories différentes : mesure persistante de disponibilité, renforcement des contrats API, qualification des futurs signaux et étude d'évolution de dépendance. La dernière reste une étude, précisément pour ne pas transformer une hypothèse technique en changement de production non justifié.

8.5 Preuves à relier à l'attendu

Attendu	Élément du dépôt	Vérification
Amélioration argumentée	Fiches de spity/improvements/ avec source, bénéfice, coûts et risques	<code>npm run improvements:check</code>
Décision réaliste	Politique de score, rôles et cadence mensuelle	<code>IMPROVEMENT_MANAGEMENT.md</code> et workflow <code>improvement-review.yml</code>
Résultat mesurable	Indicateur de départ, cible et règle de clôture	C431-02 et contrôleur de backlog
Cas d'échec couvert	Score erroné, ordre incohérent et donnée personnelle rejetés	<code>npm run improvements:exercise</code> , preuve C431-03

La simulation support utilisée comme exemple de futur signal est clairement déclarée. Elle sert à concevoir la collecte, non à affirmer une satisfaction ou un retour utilisateur réellement reçu.

9. C4.3.2 - Établir le journal des versions

9.0 Ce que la compétence demande et le problème de traçabilité

Le journal de versions attendu doit permettre de savoir ce qui a été publié, corrigé et réellement déployé. Un changelog seul décrit des évolutions produit, mais il ne suffit pas à identifier le binaire exécuté dans un environnement. Spity associe donc chaque entrée à une version SemVer, une révision Git complète, un statut explicite, des correctifs documentés et des preuves.

La règle essentielle est de ne pas confondre trois événements : la publication d'une version, la validation d'un candidat et l'observation de la production. Cette séparation est particulièrement importante dans un projet où un staging CI peut réussir avant qu'une promotion de production ne soit autorisée.

9.1 Source de vérité et statuts

Le répertoire `spity/release-journal/` remplace la note manuelle par trois fiches JSON contrôlées. Une `release published` atteste le tag et la publication GitHub ; une fiche `observed-production` ne compte comme déployée que si la santé renvoie ok avec exactement la version et le SHA annoncés ; un `candidate` contient un résultat CI ou staging, mais reste explicitement hors du journal des déploiements effectifs. `CHANGELOG.md` explique les changements produit, tandis que le registre relie ces changements à l'identité d'un logiciel exécuté et à sa preuve.

Les trois états retenus sont factuels : `v0.1.0` a été publiée le 20 juillet 2026 sur le commit `0bdd4e7` ; l'instance `jury 0.1.0-jury` à la révision `49c4ea0` a été observée saine le 13 août ; le correctif de contraste `e3784b7` est un candidat CI vert, volontairement non déclaré déployé. Cette distinction évite de présenter une validation de pipeline comme une promotion de production.

9.2 Correctifs et évolutions documentés

Chaque fiche contient les fonctionnalités, les correctifs, leur type, leur résumé, leur documentation, les risques utiles, le rollback, l'historique attribué et les preuves. La version effectivement observée rattache le correctif de dépendances runtime à `CHANGELOG.md`, à la preuve de promotion `C422-01` et à la capture de santé `C412-02`. Le candidat d'accessibilité lie de la même manière la fiche d'anomalie, sa reproduction et sa validation, sans prétendre à une mise en production.

Le validateur `scripts/check-release-journal.mjs` exige des identifiants stables, une version SemVer, un SHA complet, des chemins internes ou URLs HTTPS lisibles, et refuse secrets, jetons, e-mails et adresses privées. Il rejette également un correctif déployé sans documentation, une preuve hors dépôt et toute différence entre l'identité de la fiche et la santé observée.

9.3 Cadence, responsabilité et contrôle

La commande `npm run releases:check` rejoint la porte de qualité et la validation de release : pour un tag, cette dernière exige aussi une fiche `candidate` ou `published` portant la même version. Le workflow `Release journal` s'exécute à chaque modification concernée, chaque premier jour du mois et conserve le rapport 90 jours. `npm run releases:exercise` accepte le registre sain puis refuse en mémoire un candidat CI promu sans santé, un correctif sans documentation et une révision de santé incohérente. L'exercice ne contacte ni production, ni base, ni LXC.

Le mainteneur ajoute les changements et le rollback ; le responsable de release vérifie la chronologie, les preuves et les corrections ; la supervision atteste l'observation de production. Une future promotion devra donc être inscrite après son contrôle post-déploiement, jamais au seul passage de la CI.

9.4 Cycle de vie d'une version et lecture devant le jury

Le journal se lit comme une chaîne de preuves :

- le mainteneur documente les fonctionnalités, corrections, risques et rollback associés à une version ;

- une publication GitHub fournit un tag et une version published ;
- la CI ou le staging peut créer une entrée candidate, qui reste explicitement hors production ;
- après une promotion autorisée, la supervision contrôle l'URL de santé, la version et le SHA ;
- seulement cette observation permet de créer ou mettre à jour une entrée observed-production ;
- le contrôleur vérifie que les identités, preuves, liens documentaires et règles de confidentialité restent cohérents.

Le cas de contraste validé en CI est volontairement maintenu au statut candidate. Il démontre une correction et des contrôles verts, mais ne permet pas de déclarer une mise en production. À l'inverse, l'instance jury est décrite avec les valeurs réellement observées par la sonde. Le journal décrit donc la réalité opérationnelle plutôt qu'une projection de l'état souhaité.

9.5 Preuves et critères de qualité

Critère	Preuve associée	Point de contrôle
Identité de version	Fiches spity/release-journal/ et C432-02	SemVer et SHA complet obligatoires.
Correctifs documentés	CHANGELOG.md, fiches d'incident et liens de documentation	Un correctif déployé sans documentation est refusé.
Statut de déploiement honnête	Santé observée, candidate CI et publication séparées	Une candidate ne devient pas production par défaut.
Continuité et rollback	Procédures et historique attribué	Chaque changement reste explicable et réversible.
Robustesse du registre	C432-03	Identité incohérente, preuve hors dépôt et promotion injustifiée sont rejetées.

Les commandes `npm run releases:check` et `npm run releases:exercice` permettent de rejouer l'audit du journal et ses cas de rejet.

10. C4.3.3 - Collaborer avec le support

10.0 Ce que la compétence demande et l'organisation retenue

Collaborer avec le support consiste à rendre visible le passage entre un problème vécu dans le produit et une résolution technique. Le support apporte le contexte fonctionnel, l'impact et les critères de réussite ; le mainteneur apporte l'analyse, la cause, le correctif, les limites et les validations. Les deux contributions doivent rester distinguables afin que la résolution soit comprise autant par le métier que par la technique.

Spity formalise ce dialogue dans un registre versionné. Une transmission n'est pas une conversation libre impossible à vérifier : elle contient le rôle émetteur, le rôle destinataire, le contenu utile, la date, les critères de validation et les preuves. La confidentialité est contrôlée au même niveau que les autres registres.

10.1 Registre et cycle de collaboration

Le registre `spity/support-collaborations/` rend la collaboration support/mainteneur contrôlable au lieu de la limiter à un récit. Chaque fiche `SPITY-SUP-YYYY-NNNN` lie un contexte anonymisé, une anomalie `SPITY-INC`, les critères d'acceptation fonctionnels, la cause racine, le correctif, les transmissions entre rôles, la validation support et les preuves. Son cycle strict est `open`, `technical-analysis`, `awaiting-support-validation`, puis `closed`.

10.2 Cas de collaboration présenté

La fiche `SPITY-SUP-2026-0001` présente le problème de contraste des états vides. Le support niveau 1 simulé décrit la base vierge, les écrans concernés, l'impact P2 et trois critères de clôture. Le mainteneur niveau 2 confirme la priorité, isole l'héritage de couleurs de `EmptyState`, rend le composant autonome et transmet ses validations `Lighthouse`, `Playwright` et `CI`. Le support reçoit ensuite la cause, le correctif et l'absence de preuve de production, puis rejoue les critères dans le scénario contrôlé avant de clôturer.

10.3 Vérification du protocole et confidentialité

`npm run support:check` refuse une fiche sans déclaration explicite de simulation, sans escalade support vers mainteneur, sans retour d'expertise, sans validation support à la clôture, avec une preuve hors dépôt/non HTTPS ou avec une donnée sensible. `npm run support:exercise` vérifie en mémoire le cas sain et ces quatre échecs. Le workflow mensuel `Support collaboration` conserve le rapport 90 jours. La simulation est explicitement déclarée : aucun retour client humain et aucun déploiement de production ne sont revendiqués.

10.4 Déroulé de résolution et éléments à présenter

Étape	Support / rôle fonctionnel	Mainteneur / rôle technique	Preuve
Ouverture	Décrit le contexte, l'impact et les critères attendus	Qualifie la priorité et relie l'incident	Fiche <code>SPITY-SUP-2026-0001</code> et <code>C433-01</code>
Analyse	Clarifie le comportement attendu	Identifie l'héritage de couleurs et propose le correctif	Transmission d'expertise et fiche d'incident
Validation	Rejoue les critères de clôture dans le scénario déclaré	Transmet <code>Lighthouse</code> , <code>Playwright</code> , <code>CI</code> et les limites de déploiement	<code>C433-02</code> et preuves du correctif
Clôture	Valide la résolution fonctionnelle simulée	Conserve la chronologie et les liens de preuve	<code>C433-03</code> et <code>npm run support:check</code>

La compétence ne prétend pas qu'un client réel a validé le produit. Elle prouve un protocole de collaboration complet, contrôlé et réutilisable, avec une simulation explicitement annoncée conformément à la modalité autorisée par le référentiel.

11. Protection des données et sécurité des preuves

Les preuves contiennent uniquement des URLs publiques, SHA Git, versions, résultats de tests et comptes temporaires. Elles excluent `.env.local`, mots de passe, jetons, IP privées, exports MariaDB et données personnelles. Les rapports sont versionnés avec un manifeste SHA-256.

Les actions destructives restent interdites dans les procédures courantes : aucune suppression de volume, aucun `npm audit fix --force`, aucun déploiement sans sauvegarde et aucune réécriture de l'historique Git.

12. Conclusion

Les sept compétences franchissent désormais la définition renforcée de terminé : mécanisme réel ou simulation clairement déclarée, automatisation, cas d'échec testés, preuves reproductibles et exploitation décrite. La revue transversale automatise ce constat : les sept sources, les preuves, les registres et le manifeste doivent rester cohérents. C4.3.3 complète cette chaîne par un registre de collaboration qui sépare rigoureusement le contexte fonctionnel, l'expertise technique, la validation support et le statut réel de déploiement.

Le principal risque ouvert n'est pas masqué : la production est saine mais en retard sur main. La prochaine action opérationnelle est une release versionnée autorisée, pas un déploiement improvisé. Cette transparence garantit que le dossier décrit l'état réel du logiciel.

13. Index des annexes

Annexe	Critère	Contenu
A1	C4.1.1	spity/MAINTENANCE.md
A2	C4.1.1	preuves/B4-C411-01-audit-dependances-2026-08-13.json
A3	C4.1.1	preuves/B4-C411-02-decision-maintenance-2026-08-13.md
A4	C4.1.1	preuves/B4-C411-03-controle-dependances-2026-08-13.json
A5	C4.1.2	spity/OBSERVABILITY.md
A6	C4.1.2	preuves/B4-C412-01-historique-supervision-2026-08-13.json
A7	C4.1.2	preuves/B4-C412-02-sante-production-2026-08-13.json
A8	C4.1.2/C4.2.1	preuves/B4-C412-03-exercice-alerte-2026-08-13.json
A8b	C4.1.2	preuves/B4-C412-04-slo-supervision-2026-08-13.json
A9	C4.2.1	preuves/B4-C421-01-fiche-anomalie-accessibilite-2026-08-13.md
A10	C4.2.1	preuves/B4-C421-02-anomalie-derive-production-2026-08-13.md
A10b	C4.2.1	spity/INCIDENT_MANAGEMENT.md et spity/incidents/
A10c	C4.2.1	preuves/B4-C421-03-registre-anomalies-2026-08-13.json
A10d	C4.2.1	preuves/B4-C421-04-exercice-registre-2026-08-13.json
A11	C4.2.2	preuves/B4-C422-01-correctif-et-ci-2026-08-13.json
A12	C4.2.2	preuves/B4-C422-02-traitement-correctif-ci-cd-2026-08-13.md
A12b	C4.2.2	preuves/B4-C422-03-exercice-verification-deploiement-2026-08-13.json
A12c	C4.2.2	spity/RELEASE_VERIFICATION.md et scripts/verify-deployment.mjs
A12d	C4.2.2	preuves/B4-C422-04-staging-verifie-2026-08-13.json
A13	C4.3.1	preuves/B4-C431-01-recommandations-2026-08-13.md
A13b	C4.3.1	preuves/B4-C431-02-registre-ameliorations-2026-08-13.json
A13c	C4.3.1	preuves/B4-C431-03-exercice-revue-ameliorations-2026-08-13.json
A13d	C4.3.1	spity/IMPROVEMENT_MANAGEMENT.md et spity/improvements/
A14	C4.3.2	preuves/B4-C432-01-journal-versions-deployees-2026-08-13.md
A14b	C4.3.2	preuves/B4-C432-02-registre-versions-2026-08-13.json
A14c	C4.3.2	preuves/B4-C432-03-exercice-journal-versions-2026-08-13.json
A14d	C4.3.2	spity/RELEASE_JOURNAL.md, release-journal/ et check-release-journal.mjs
A15	C4.3.3	spity/SUPPORT.md et preuves/B4-C433-01-collaboration-support-2026-08-13.md
A15b	C4.3.3	preuves/B4-C433-02-registre-collaboration-support-2026-08-13.json
A15c	C4.3.3	preuves/B4-C433-03-exercice-collaboration-support-2026-08-13.json
A15d	C4.3.3	spity/support-collaborations/ et check-support-collaborations.mjs
A16	Intégrité	preuves/MANIFEST.sha256
A17	Revue finale	REVUE_FINALE_BLOC_04.md et preuves/B4-REVUE-FINALE-01-audit-transversal-2026-08-13.json

Les sources complémentaires regroupent les workflows et formulaires sous `.github/`, les politiques et scripts de maintenance sous `spity/`, les guides de déploiement, de release, de journal et d'amélioration, ainsi que le référentiel officiel archivé. Elles sont toutes reliées au manifeste d'intégrité.